

PARADIGMAS DE LINGUAGENS DE PROGRAMAÇÃO

Prof. Rogerio Mandelli

ESCOLHA VIVER
MAIS EXPERIÊNCIAS.

Aula 03 | 13 mai 2026

UVA ●
A sua melhor escolha.



Aula 03 – Paradigma Estruturado (C e Python)

Principais Estruturas de Controle



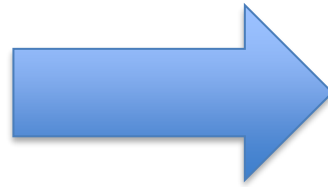
Objetivos

- Compreender o **Paradigma Estruturado** e suas principais estruturas de controle.
- Aplicar os conceitos de **sequência, decisão e repetição** em C e Python.
- Entender a importância da **modularização, funções e procedimentos**.
- Desenvolver uma **Calculadora Modular** e discutir eficiência e projeto de linguagens.

Conexão Aula 2 → Aula 3

Aula 2:

- implementação das linguagens
- compilação/interpretação
- JVM
- memória e escopo



Aula 3:

- organização do programa
- controle do fluxo
- modularização
- decomposição de problemas

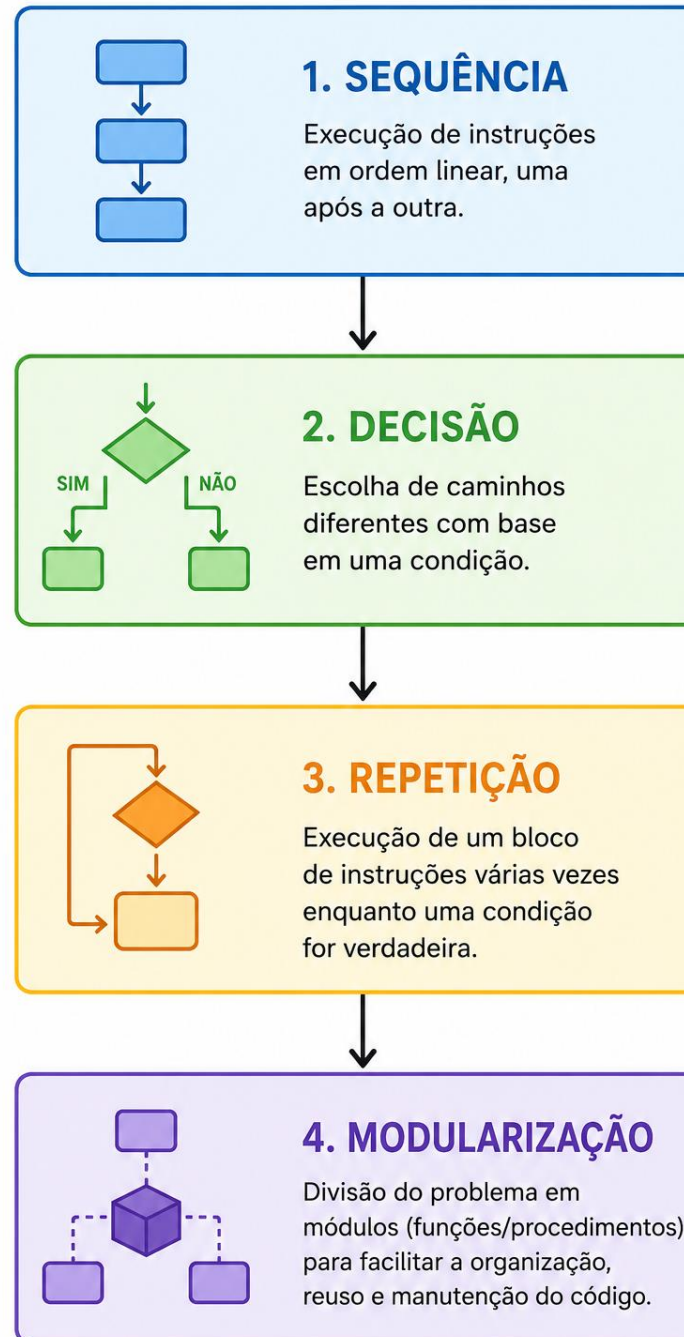
Introdução ao Paradigma Estruturado

- **Definição:** Paradigma que organiza o programa em **blocos lógicos e estruturas de controle** previsíveis, reduzindo o uso de comandos GOTO.
- **Princípios:**
 - Clareza e legibilidade
 - Modularização
 - Controle de fluxo bem definido
- **Influência:** base do pensamento algorítmico moderno e precursor da Programação Orientada a Objetos.

Introdução ao Paradigma Estruturado

- **Características Principais (Os Três Pilares)**
 - **Base teórica do Paradigma Estruturado:** *qualquer* algoritmo computável, não importa o quão complexo, pode ser implementado usando apenas três estruturas de controle fundamentais.
 - Isso é o que caracteriza o paradigma:
 - 1. Sequência:** A regra mais básica. O código é executado linha por linha, de cima para baixo, sem saltos inesperados.
 - 2. Seleção (ou Decisão):** Permite que o programa escolha um entre dois ou mais caminhos, baseado em uma condição.
Exemplos: `if`, `else`, `else if`, `switch/case`.
 - 3. Iteração (ou Repetição):** Permite que um bloco de código seja executado múltiplas vezes, enquanto uma condição for verdadeira (ou por um número definido de vezes).
Exemplos: `while`, `for`, `do/while`.

Os Pilares do Paradigma Estruturado



Estruturas de Controle

1. Sequência

- Estrutura mais **fundamental**.
- Computador **executa uma instrução**, depois a **próxima**, depois a próxima, de **cima para baixo**.
- **O que é:** Execução de instruções em ordem linear, uma após a outra, sem desvios.
 - **Regra:** A instrução na linha 5 só executa depois da linha 4.
 - **Analogia:** Uma receita de bolo. Você *precisa* misturar os ingredientes (linha 1) *antes* de colocar no forno (linha 2). A ordem importa.

Estruturas de Controle

- **Sequência - Exemplo Prático (Comum às 3 linguagens):**
 - Cálculo da área de um retângulo.

// C

```
int largura = 10;           // 1ª instrução
int altura = 5;             // 2ª instrução
int area = largura * altura; // 3ª instrução
printf("%d", area);         // 4ª instrução
```

Estruturas de Controle

- **Sequência - Exemplo Prático (Comum às 3 linguagens):**
 - Cálculo da área de um retângulo.

Python

largura = 10

1ª instrução

altura = 5

2ª instrução

area = largura * altura

3ª instrução (depende da 1 e 2)

print(area)

4ª instrução (depende da 3)

Introdução ao Paradigma Estruturado

2. Decisão (ou Seleção)

- Programa deixa de ser uma "linha reta" e ganha "bifurcações".
- Estrutura que permite ao programa escolher qual **caminho seguir** baseado em uma **condição** ("verdadeira" ou "falsa").
- **O que é:** Escolher um bloco de código para executar entre várias alternativas.
 - **Analogia:** Um GPS. "Se houver trânsito na via principal (condição), recalcule a rota (caminho A), senão, siga em frente (caminho B)."
 - **Palavras-chave:** `if`, `else`, `else if`, `switch`.

Estruturas de Controle

- **Decisão - Exemplo Prático:**
 - Verificar se uma nota é suficiente para aprovação (média 7.0).

```
// C
float nota = 8.5;

if (nota >= 7.0)
{
    printf("Aprovado!"); // Caminho A
}
else
{
    printf("Reprovado."); // Caminho B
}
```

Estruturas de Controle

- **Decisão - Exemplo Prático:**
 - Verificar se uma nota é suficiente para aprovação (média 7.0).

```
# Python
```

```
nota = 9.0
```

```
if nota >= 7.0:
```

```
    print("Aprovado!")    # Caminho A
```

```
else:
```

```
    print("Reprovado.")    # Caminho B
```

Estruturas de Controle

- **Decisão - Exemplo Prático:**
 - Verificar se uma nota é suficiente para aprovação (média 7.0).

```
// Java
double nota = 6.0;

if (nota >= 7.0)
{
    System.out.println("Aprovado!");    // Caminho A
} else if (nota >= 5.0)
{
    System.out.println("Recuperação."); // Caminho B
} else
{
    System.out.println("Reprovado.");    // Caminho C
}
```


Estruturas de Controle

3. Repetição (ou Iteração)

- Estrutura que dá "**poder**" ao computador.
- Permite que uma **mesma sequência** de código seja **executada várias vezes**, sem necessidade de redefini-la.
- **O que é:** Repetir um bloco de código enquanto uma condição for verdadeira (ou por um número específico de vezes).
 - **Analogia:** Apertar os parafusos de uma roda. "Para cada parafuso (iteração), aperte-o (bloco de código)."
 - **Palavras-chave:** `for`, `while`, `do...while`.

Estruturas de Controle

- **Repetição Contada - Exemplo Prático:**
 - **Loop for:** Usado quando sabemos (ou podemos calcular) o número de repetições.
 - Sintaxe: varia muito entre as linguagens, mas a ideia é a mesma.

```
// C (Sintaxe clássica: inicialização; condição; incremento)
int i;
for (i = 1; i <= 3; i++)
{
    printf("%d\n", i);
}
printf("Acabou.\n");
```

Estruturas de Controle

- **Repetição Pré-Condicionada - Exemplo Prático:**
 - **Loop while:** Testa a condição **antes** de executar o bloco.
 - Pode executar 0 vezes se a condição já for falsa de início.
 - **Exemplo:** Contar de 1 a 3.

```
# Python
i = 1
while i <= 3:
    print(i)
    i = i + 1 # Importante: alterar a condição!
print("Acabou.")
```

Estruturas de Controle

- **Repetição `for` iterável – Exemplo Prático:**
 - Em **Python**, o **`for`** normalmente percorre elementos de uma coleção. **Exemplo:** Mostrar na tela os conteúdos da lista **`nomes`** []

```
# Python
```

```
nomes = ["Ana", "Carlos", "João"]
```

```
for nome in nomes:  
    print(nome)
```

Estruturas de Controle

Repetição

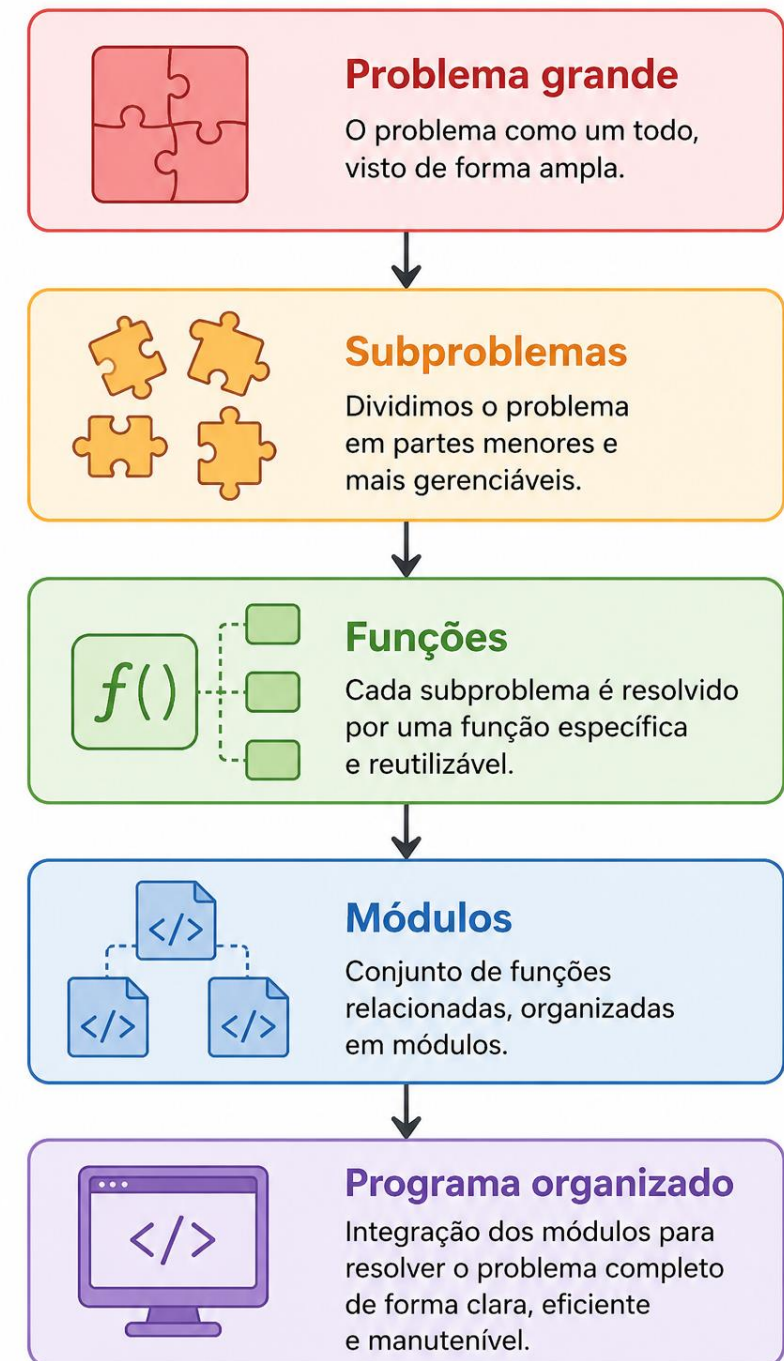
Estrutura	Quando usar	Ideia central	Exemplo típico
for	Quando sabemos, ou conseguimos calcular, quantas vezes repetir	repetição controlada por contador	repetir 10 vezes; percorrer uma lista
while	Quando não sabemos previamente quantas repetições ocorrerão	repetir enquanto uma condição for verdadeira	continuar até o usuário digitar uma opção válida
do/while	Quando o bloco precisa executar pelo menos uma vez	testar a condição depois da primeira execução	mostrar um menu antes de verificar se o usuário quer sair

Comparação Rápida — Estruturas em C e Python

Aspecto	C	Python
Blocos	chaves { }	indentação
Tipagem	explícita	dinâmica
Repetição	for clássico	for iterável
Decisão	if/switch	if/elif
Verbosidade	maior	menor

Modularização: Decomposição do Problema

- Ideia: **quebrar** o problema grande em **sub-programas menores**.
- Cada sub-programa vira um "**módulo**" (uma **função** ou um **procedimento**).



Por que Modularizar?

- **Facilita manutenção e reuso**
 - Cada módulo é uma parte independente do programa, com função bem definida.
 - Se for necessário corrigir um erro ou fazer uma melhoria, apenas aquele módulo precisa ser alterado.
 - Um módulo bem projetado pode ser reutilizado em outros programas, economizando tempo e esforço.

Por que Modularizar?

- **Reduz complexidade cognitiva**
 - Programas grandes são difíceis de entender como um todo.
 - A modularização divide o problema em partes menores e compreensíveis.
 - O programador pode focar em uma parte de cada vez, entendendo sua função e comportamento isoladamente.

Por que Modularizar?

- **Permite divisão entre equipes**
 - Cada membro (ou grupo) pode desenvolver um módulo diferente de forma paralela.
 - A integração ocorre depois, facilitada por interfaces bem definidas (funções e parâmetros).

Por que Modularizar?

Benefício	Impacto
Reuso	menos repetição
Clareza	código mais compreensível
Manutenção	correções localizadas
Escalabilidade	crescimento organizado

Funções e Procedimentos

- **Funções**

- Uma **função** é um bloco de código que **realiza uma tarefa específica e devolve um valor** para quem a chamou.
- Ela tem **parâmetros (entradas)**, um **processamento interno**, e um **valor de retorno (saída)**.

```
float soma(float a, float b)
{
    return a + b;
}
```

Funções e Procedimentos

- **Procedimentos (ou sub-rotinas)**
 - Um **procedimento** (ou sub-rotina) também é um bloco de código que **realiza uma ação específica**, mas **não devolve um valor**.
 - Ele é usado quando queremos **executar comandos**, e não **calcular um resultado**.
 - Em C e Java, um procedimento é uma função declarada com o tipo de retorno `void` (isto é, *vazio*).

```
# include <stdio.h>

void exibirMensagem()
{
    printf("Bem-vindo ao programa!\n");
}

int main()
{
    exibirMensagem();
}
```

Tratamento de funções em C, Python e Java

- **Modularização nas Linguagens – C, Python e Java:**
 - **C e Java:** Mantêm a distinção clara.
 - Métodos/funções que retornam `void` são procedimentos.
 - Métodos/funções que retornam um tipo (`int`, `double`) são funções.
 - **Python:** Simplificou. **Tudo é uma função** (definida com `def`).
 - Uma função que *age* como um **procedimento** é aquela que você **não escreve** `return`.
 - Tecnicamente, mesmo essa função "procedimento" retorna um valor especial: `None` (nulo).

Tratamento de funções em C, Python e Java

Linguagem	Tratamento
C	função/procedimento separados (<code>void</code>)
Java	métodos <code>void</code>
Python	tudo é função
Python sem <code>return</code>	retorno implícito <code>None</code>

```
def mensagem():  
    print("Olá")
```

```
x = mensagem()  
print(x)
```

Saída:
Olá
None

Funções em C e Python

```
float soma(float a, float b)
{
    return a + b;
}
```

```
int main()
{
    float x, y;
    printf("Digite dois valores: ");
    scanf("%f %f", &x, &y);
    printf("Resultado: %.2f", soma(x, y));
}
```


Funções em C e Python

```
# Procedimento (retorna None implicitamente)
def imprimir_resultado(valor):
    print(f"O resultado é: {valor}")

# Função (retorna um int)
def somar(a, b):
    return a + b

# --- Principal ---
resultado = somar(10, 20)
imprimir_resultado(resultado)
```

Escopo e Modularização

`ex_31_P_escopo-lexico.py`

Objetivo

- Conectar:
 - funções;
 - escopo local;
 - estado;
 - encapsulamento.
- **Mensagem central**
 - Funções também isolam contexto e estado.

Mutabilidade e Efeitos Colaterais

`ex_32_P_conteudo-var.py`

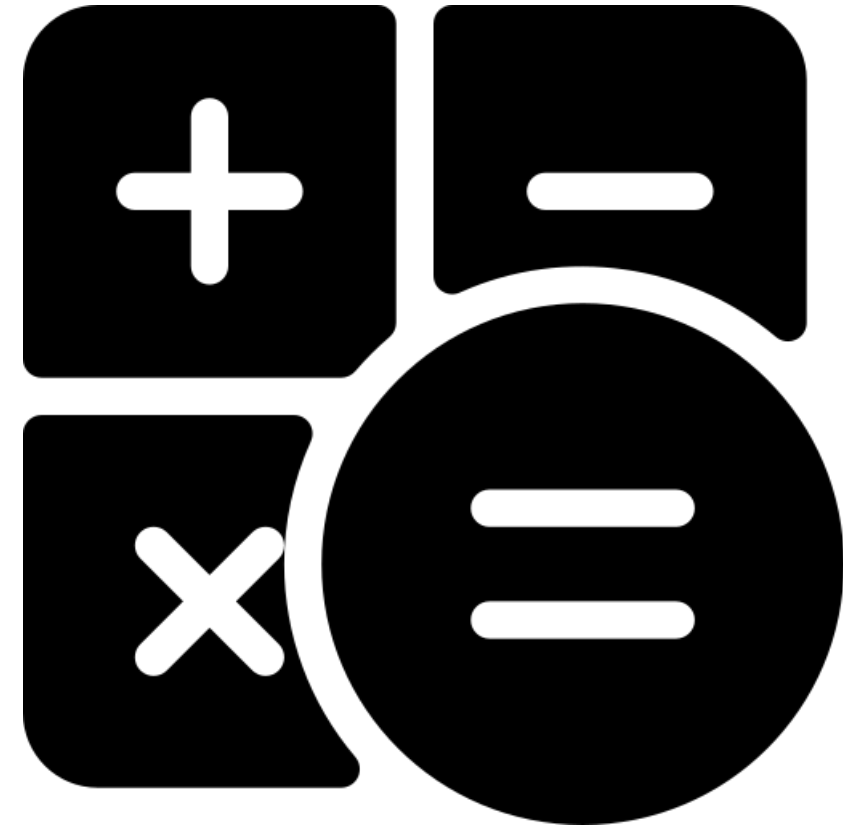
Objetivo

- Mostrar:
 - mutabilidade
 - mudança de estado
 - impacto em modularização
- modularização também envolve controle de estado e comportamento dos dados.

Projeto Incremental: Calculadora Modular

Objetivo

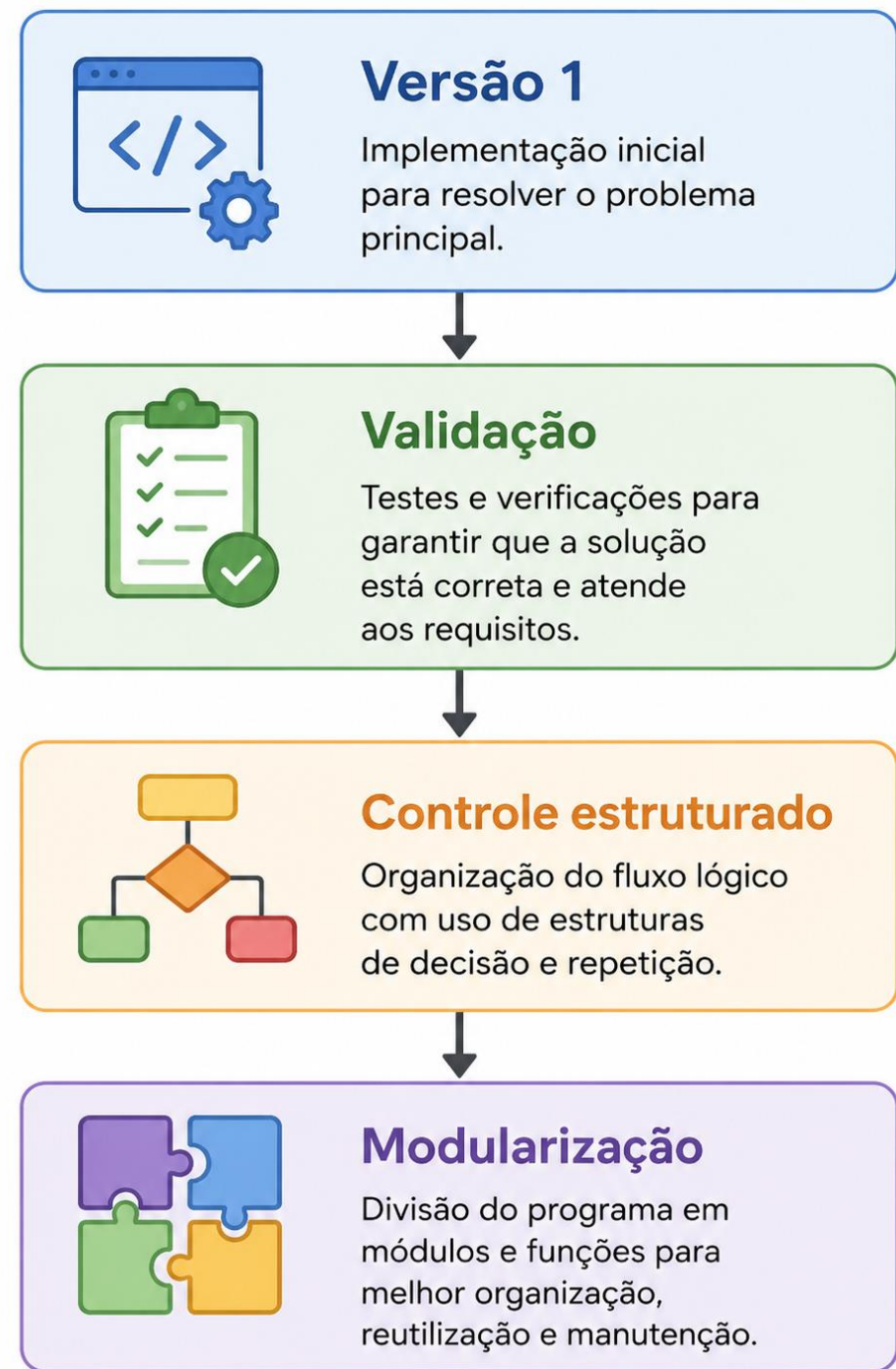
- cada versão adiciona organização
- não é apenas “mais código”
- é evolução arquitetural



Projeto Incremental: Calculadora Modular

Versões em C

Versões em Python



Versão 1C - Decomposição em Funções

Problema:

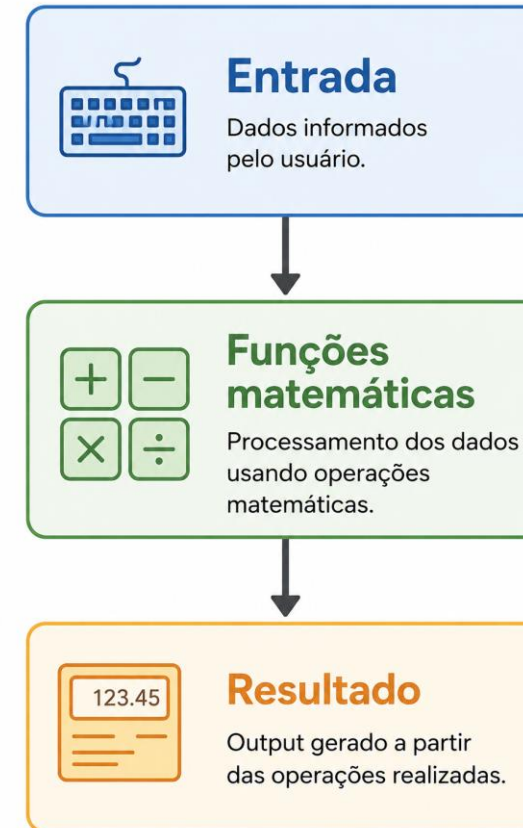
- realizar operações matemáticas

Solução estruturada:

- dividir cada operação em uma função específica

`ex_33_C_calculadora_v1.c`

Função	Responsabilidade
<code>soma()</code>	adicionar
<code>subtrai()</code>	subtrair
<code>multiplica()</code>	multiplicar
<code>divide()</code>	dividir



→ **Paradigma estruturado reduz complexidade dividindo o problema em partes menores**

Versão 1C - Decomposição em Funções

ex_33_C_calculadora_v1.c

`</> C`

```
float soma(float a, float b)
{
    return a + b;
}
```

`</> Python`

```
def soma(a, b):
    return a + b
```

Versão 2C - Validação e Robustez

Problema:

- Nem toda entrada do usuário é válida

`ex_33_C_calculadora_v2.c`

Conceito	Papel
<code>if</code>	valida condição
retorno especial	evita falha
validação	aumenta robustez
previsibilidade	reduz erros

↗ C

```
float divide(float a, float b)
{
    if (b == 0)
    {
        printf("Erro: divisão por zero.\n");
        return 0;
    }

    return a / b;
}
```

↗ Python

```
def divide(a, b):
    if b == 0:
        print("Erro")
        return None
    return a / b
```

→ **Programas estruturados devem controlar situações inválidas de forma previsível**

Versão 3C - Controle Estruturado do Programa

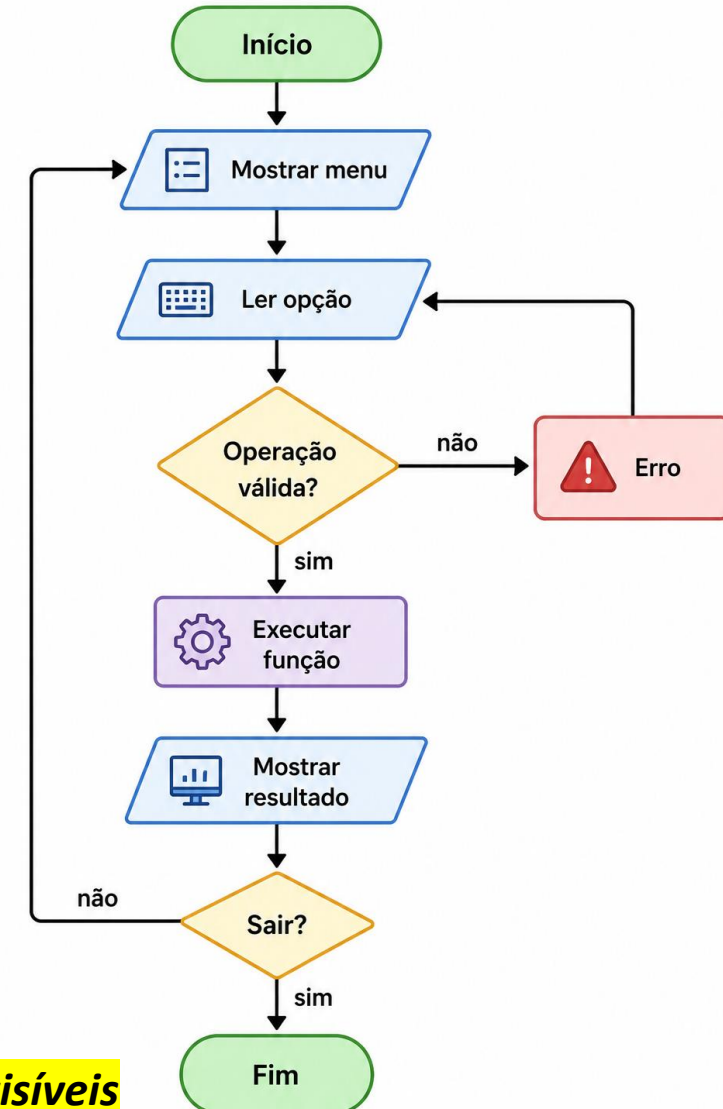
O programa passa a controlar:

- entrada
- decisão
- repetição
- encerramento

`ex_33_C_calculadora_v3.c`

Estrutura	Papel
<code>while / do-while</code>	laço principal
<code>if / switch</code>	seleção
funções	modularização
menu	controle do fluxo

→ ***O paradigma estruturado organiza a execução do programa em ciclos previsíveis***



Versão 4C - Modularização em Arquivos

`ex_33_C_calculadora_v4_main.c`

Na versão 3:

- funções matemáticas e fluxo principal estavam no mesmo arquivo
- o `main()` concentrava menu, leitura, decisão e saída
- havia repetição de código

Na versão 4:

- operações matemáticas vão para `ops.c`
- interface e leitura vão para `ui.c`
- `main.c` coordena a execução
- arquivos `.h` declaram o que cada módulo oferece

→ **Modularizar é separar responsabilidades para tornar o programa mais claro, reutilizável e fácil de manter**

Versão 4C - Modularização em Arquivos

`ex_33_C_calculadora_v4.c`

`main.c`

coordena o programa

|

|— chama ui.h / ui.c

|

menu, leitura, mensagens

|

|— chama ops.h / ops.c

soma, subtrai, multiplica, divide

`ex_33_C_calculadora_v4_ui.h`

`ex_33_C_calculadora_v4_ui.c`

`ex_33_C_calculadora_v4_ops.h`

`ex_33_C_calculadora_v4_ops.c`

Versão 4C - Modularização em Arquivos

`ex_33_C_calculadora_v4_main.c`

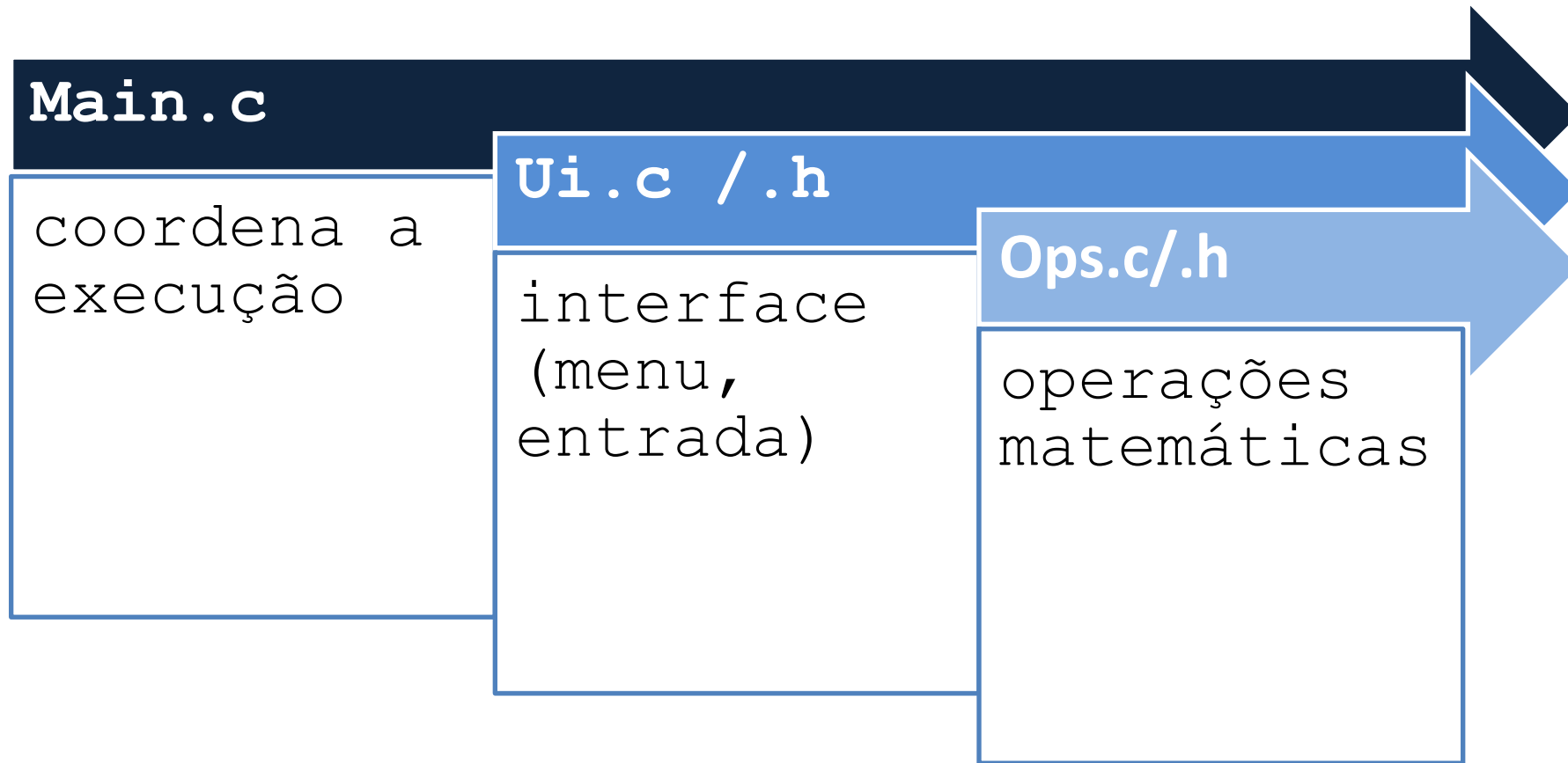
Aspecto	Versão 3	Versão 4
Organização	um arquivo principal	múltiplos módulos
<code>main()</code>	faz quase tudo	coordena
Operações	no mesmo arquivo	em <code>ops.c</code>
Interface	misturada ao fluxo	em <code>ui.c</code>
Manutenção	mais difícil	mais localizada
Reuso	baixo	maior

Versão 4C - Modularização em Arquivos (arquivos .c e .h)

`ex_33_C_calculadora_v4_main.c`

- **Enunciado:**
 - Separar a calculadora em módulos independentes aplicando modularização física e encapsulamento.
- **Arquivos:**
 - `ops.c` / `ops.h`: funções matemáticas.
 - `ui.c` / `ui.h`: interface e leitura de dados.
 - `main.c`: laço principal e coordenação.
- **Compilação:**
 - `gcc main.c ui.c ops.c -o calculadora`
- **Conceitos:**
 - Modularização física.
 - Encapsulamento e reuso.
 - Include guards e protótipos de funções.

Versão 4C - Modularização em Arquivos (arquivos .c e .h)



Versão 4C - Modularização em Arquivos (`arquivos .c e .h`)

Conceito	Onde aparece	Função principal	Exemplo
.c	<code>ops.c</code> , <code>ui.c</code> , <code>main.c</code>	Implementa o código	<code>float soma(float a, float b)</code>
.h	<code>ops.h</code> , <code>ui.h</code>	Declara funções públicas	<code>float soma(float, float);</code>
Modularização física	Versão 4	Divide o código em partes independentes	<code>main.c</code> coordena <code>ops.c</code> e <code>ui.c</code>
Encapsulamento	Versão 4	Ocultar implementação interna	<code>main.c</code> usa <code>mostrar_menu()</code> sem saber seu conteúdo
Include guards	Todos os <code>.h</code>	Evitam duplicação de definições	<code>#ifndef OPS_H ...</code> <code>#endif</code>

Resumo das Versões C

Versão	Características principais	Conceitos-chave
1.0	Estrutura sequencial e funções	Funções e variáveis
2.0	Validação e clareza	Condicionais e tratamento de erro
3.0	Controle estruturado (<code>switch</code> , <code>do...while</code>)	Estruturas de controle
4.0	Modularização com múltiplos arquivos	Encapsulamento e reuso

Versão 1P – Decomposição em Funções

`ex_33_P_calculadora_v1.py`

- **Enunciado**
 - Desenvolver uma calculadora simples em Python que realiza operações básicas entre dois números.
- **Requisitos**
 - Estrutura sequencial simples.
 - Funções para cada operação: `soma`, `subtrai`, `multiplica`, `divide`.
 - Leitura de dois valores com `input()`.
 - Exibição formatada do resultado.

Versão 2P – Validação e Robustez

`ex_33_P_calculadora_v2.py`

- **Melhorias**
 - Tratamento de erro para divisão por zero.
 - Melhor formatação da saída.
 - Mensagens mais claras ao usuário.

Versão 3P – Controle Estruturado do Programa

`ex_33_P_calculadora_v3.py`

- **Enunciado**

- Reescrever a calculadora em Python com um menu interativo que permite ao usuário escolher a operação desejada e repetir o processo até optar por sair.

- **Requisitos**

- Menu com `print()` e `input()`.
 - Estrutura `while True` (laço principal).
 - Uso de `if/elif/else` para seleção da operação.
 - Opção de saída (0).

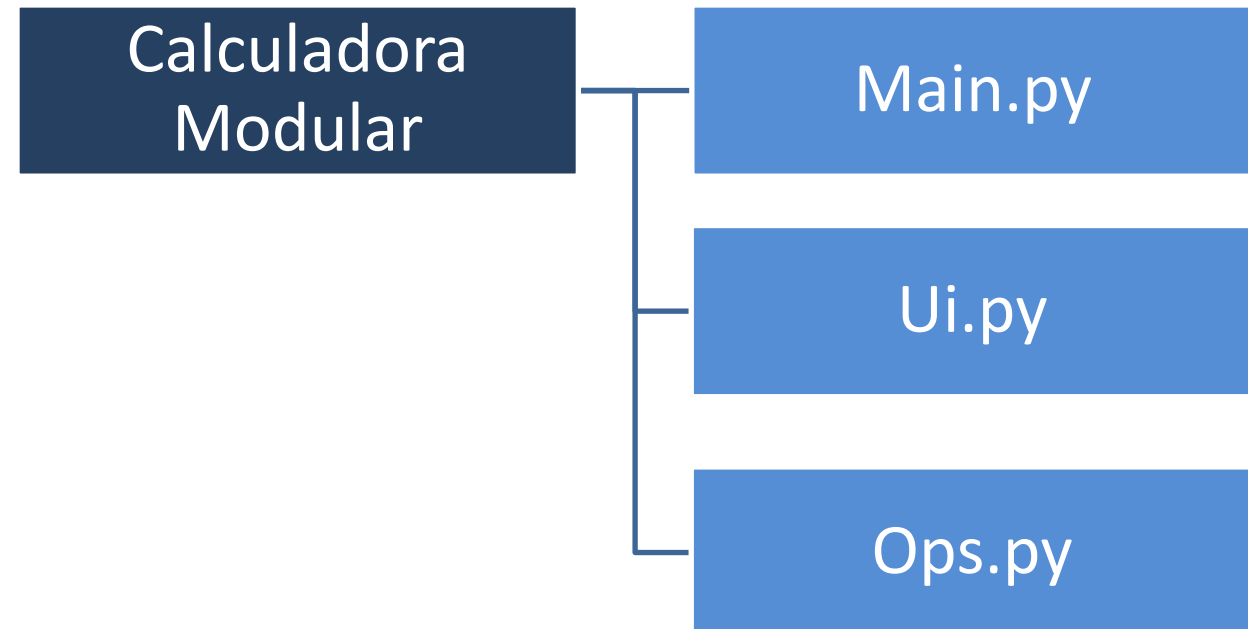
Versão 4P – Modularização em Arquivos

- **Enunciado**
 - Reorganizar a calculadora em **módulos separados** (arquivos `.py`), simulando a modularização física adotada em C.
 - Refinamento dos dados de entrada.

`ex_33_P_calculadora_v4_main.py`

`ex_33_P_calculadora_v4_ui.py`

`ex_33_P_calculadora_v4_ops.py`



Resumo das Versões Python

Versão	Descrição	Foco conceitual
1.0	Estrutura sequencial e funções simples	Funções, parâmetros e retorno
1.1	Validação e mensagens de erro	Condicionais e clareza
2.0	Menu interativo com laço principal	Estruturas de controle (<code>while</code> , <code>if/elif/else</code>)
3.0	Modularização com múltiplos arquivos	<code>import</code> , reuso e tratamento de exceções

Comparativo — Calculadora Modular em C e Python

Versão	C – Características e Estrutura	Python – Características e Estrutura	Conceitos-Chave Comuns
1.0	Programa simples com funções em um único arquivo (<code>main.c</code>). Uso de <code>scanf</code> , <code>printf</code> , e tipos explícitos (<code>float</code>).	Programa simples com funções em um único arquivo. Uso de <code>input()</code> , <code>print()</code> , e tipagem dinâmica.	Estrutura sequencial, funções e variáveis.
2.0	Adição de validação (ex: divisão por zero). Mais mensagens e legibilidade de código.	Adição de validação (divisão por zero via <code>if</code>). Melhor formatação da saída.	Condicionais, clareza e tratamento de erro.
3.0	Implementação de menu interativo com <code>switch-case</code> e <code>do...while</code> . Controle explícito de laço e fluxo.	Implementação de menu interativo com <code>if/elif/else</code> e <code>while True</code> . Controle de laço com <code>break</code> .	Estruturas de controle, repetição e seleção múltipla.
4.0	Modularização física: arquivos <code>.c</code> e <code>.h</code> (<code>main.c</code> , <code>ui.c</code> , <code>ops.c</code>). Uso de include guards e encapsulamento .	Modularização lógica: arquivos separados (<code>main.py</code> , <code>ui.py</code> , <code>ops.py</code>). Uso de import e namespaces .	Modularização, encapsulamento, reuso e separação de responsabilidades.

Síntese Geral

Aspecto	C	Python
Tipagem	Estática (exige declaração de tipo)	Dinâmica (detecta tipo automaticamente)
Entrada/Saída	<code>scanf, printf</code>	<code>input, print</code>
Estruturas de controle	<code>if, switch, for, while, do...while</code>	<code>if, elif, for, while</code>
Modularização	Arquivos <code>.c</code> + cabeçalhos <code>.h</code>	Arquivos <code>.py</code> + <code>import</code>
Compilação	Necessária (<code>gcc</code>)	Interpretação direta
Paradigma	Estruturado, imperativo	Estruturado, imperativo
Encapsulamento	Via headers (<code>.h</code>) e funções	Via módulos e namespaces
Execução	<code>gcc main.c ... -o calculadora</code> → <code>calculadora.exe</code>	<code>python main.py</code>

Conclusão

- Apesar de diferenças sintáticas e técnicas, **C e Python seguem os mesmos princípios estruturados:**
 - **Divisão em funções e módulos** (reuso e clareza).
 - **Controle de fluxo explícito** (laços, condicionais).
 - **Separação lógica e física do código** (encapsulamento).
 - **Python simplifica a execução e a legibilidade**, enquanto **C explicita o controle e a estrutura da memória**, tornando o exercício um excelente paralelo entre **linguagem de alto nível e de sistema**.

Referências

- SEBESTA, R. W. Conceitos de Linguagens de Programação, Cap. 1–3.
- TUCKER & NOONAN. Linguagens de Programação: Princípios e Paradigmas.
- Vídeo: How Compilers and Interpreters Work – Computerphile.

